

WM 2026 — Prognose-System · Gesamtdokumentatio n

2026-06-08

Inhalt

VISION — WM 2026 Prognose- & Tipp-System
PROJECT_STATUS — Aktueller Projektstand
ARCHITECTURE — Systemübersicht
ML_MODEL — Modellarchitektur & Mathematik
MODEL_EXPLANATION — Das Modell in einfachen Worten
PREDICTION_LOGIC — Schritt für Schritt
MODEL_EVALUATION — Ist die Komplexität gerechtfertigt?
DATA_SOURCES — Datenquellen & Ingestion
DATABASE_SCHEMA — PostgreSQL
API_SPEC — REST-Endpunkte
BETTING_ENGINE — Decision Layer (EV + Edge)
LIMITATIONS — Bekannte Grenzen (bewusst & dokumentiert)
DEPLOYMENT — Free-Stack (Vercel + Render + Neon)
DECISIONS — Architecture Decision Records (ADR)
ROADMAP — Verlauf & Stand
RECOVERY — Quellcode-Wiederherstellung (2026-06-07)

VISION — WM 2026 Prognose- & Tipp-System

Ziel

Ein **datengetriebenes, transparentes** Prognose- und Tipp-Tool für die Fußball-WM 2026, das von überall im Browser nutzbar ist — als persönliches Werkzeug für Tipprunden und zur Turnier-Analyse.

Leitprinzipien

1. **Reproduzierbar & datengetrieben** — keine Bauchwerte. Stärke aus `elratings.net` + live gelernt; Form aus echten Ergebnissen; klare Trennung `COMPUTED` vs. `PRIOR`.
2. **Transparenz** — jede Prognose erklärt ihre Faktoren (Elo-Differenz + Kontextfaktoren mit Beitrag und Richtung); Limitationen werden offen kommuniziert.
3. **Statistisch fundiert, nicht überkomplex** — Elo + Poisson + Dixon-Coles tragen die Vorhersage; Zusatzfaktoren bleiben bewusst klein (durch Evaluation belegt).
4. **Robust & einfach im Betrieb** — graceful degradation (ohne Redis/Odds lauffähig), deterministische Kernlogik, kostengünstiges Hosting.

Kernfunktionen

- Spiel-Prognosen (W/U/N, xG, Scoreline-Heatmap, Erklärung)
- Gruppen-Tabellen + Qualifikations-Wahrscheinlichkeiten
- Monte-Carlo-Titelchancen + Turnierbaum
- Tipprunden-Tipps (xG-Tipp, Modell-Tipp, punktoptimaler Tipp)
- Betting Decision Engine (EV + Edge gegen echte Marktquoten) — reines Analyse-Tool
- Live-Lernen: nach jedem Ergebnis aktualisieren sich Elo, Form, KO-Baum, Prognosen, Simulation

Nicht-Ziele

Kein Wettanbieter, kein Glücksspiel-Aufruf. Keine ML-Black-Box. Keine nicht-reproduzierbaren Datenquellen. Keine Skalierung über das hinaus, was ein privates Tool braucht.

Zielgruppe

Der Betreiber + Freunde/Tipprunde. Geringe Last, klarer Fokus auf Erklärbarkeit und Spaß.

PROJECT_STATUS — Aktueller Projektstand

Zuletzt aktualisiert: 2026-06-08 **Phase:** Live in Produktion (kostenloser Stack), performance-optimiert, Auto-Sync aktiv **WM-Start:** 2026-06-11

Hinweis: Die `docs/` wurden am 2026-06-07 nach einem Quellcode-Verlust vollständig neu erstellt und spiegeln den **tatsächlichen Stand des Codes**. Siehe [RECOVERY.md](#).

Live-Deployment (0 €/Monat)

Komponente	Dienst	URL
Frontend	Vercel	https://wm2026-prediction-system.vercel.app
Backend-API	Render (Free, Docker, Frankfurt)	https://wm2026-backend-phwx.onrender.com
Datenbank	Neon (Postgres, eu-central)	(intern)
Code	GitHub	github.com/annigue/wm2026-prediction-system

Details + Performance-Architektur: [DEPLOYMENT.md](#).

System (was läuft)

- **Elo-Rating** (Init aus [elratings.net](#), live via Bayesian Update nach jedem Ergebnis)
 - **Poisson + Dixon-Coles** Tormodell → $W/U/N + xG + \text{Scoreline-Verteilung}$
 - **Feature Adjustment Layer** (5 deterministische Faktoren als Elo-Delta)
 - **Context Injection Layer** (stochastische Varianz in der Simulation)
 - **Monte-Carlo-Turniersimulation** (Prod: 100.000 Runs, Hintergrund-Task)
 - **KO-Bracket-Resolver** (füllt KO-Spiele aus realen Ergebnissen)
 - **Betting Decision Engine** (EV + Edge + NO_BET) mit echter Odds-Integration
 - **Tipping Engine** (punktoptimaler Exakt-Tipp) + **Tipp-vs-Ergebnis-Auswertung** (Kicktipp-Punkte)
 - **Automatischer Ergebnis-Sync** (CI-Cron → `/admin/auto-update`, idempotent)
 - **Form Engine v2** (Punkte + Tore + Recency, Single Source of Truth)
 - FastAPI-Backend + Next.js-14-Frontend + PostgreSQL
-

Automatischer Datenfluss (kein manuelles Eintragen nötig)

```
GitHub Actions (sync-results.yml, alle 30 min)
→ POST /admin/auto-update (ADMIN_TOKEN)
  → sync_all: echte Ergebnisse von der WM-API in die DB
  → Elo idempotent für NEU beendete Spiele (nur Spiele ohne
```

Elo-Eintrag)
→ Hintergrund: Form → KO-Bracket → Prognosen → Cache →
Simulation
GitHub Actions (keepalive.yml, alle 10 min) → /health (gegen
Render-Cold-Start)

Manuelles Eintragen via `POST /matches/{id}/result` bleibt als Fallback möglich.

Performance-Architektur (Interface reagiert schnell)

- **Frontend ISR** (`revalidate = 60`): Listenseiten aus Vercel-Cache (~0.1 s).
- **generateStaticParams**: alle Spiel-/Team-Detailseiten beim Build vorgerendert → erster Aufruf sofort.
- **Backend in Frankfurt** (co-located mit Neon) → keine US↔EU-Latenz pro Query (Detail 4.5 s → 0.12 s).
- **Verschlanke Queries** (keine volle Prognose-JSONB/elo_history in Listen) + **In-Process-Cache** für die Projektion.
- **Keep-Alive-Ping** gegen Cold-Start. Die 100k-Simulation läuft im Hintergrund und blockiert das Interface nicht.

Dateistruktur (Backend `backend/app/`)

Datei	Rolle
<code>services/elo_model.py</code>	Elo: expected_score, update, goal_diff_multiplier
<code>services/poisson_model.py</code>	Poisson + Dixon-Coles ($p=-0.13$), PredictionResult
<code>services/feature_adjuster.py</code>	5-Faktor-Adjustment (Form, Markt, Höhe, Reise, Rest-Days)
<code>services/context_modifier.py</code>	Simulations-Varianz (Form, Markt, KO-Druck, Umwelt-Stress)
<code>services/prediction_engine.py</code>	Orchestriert Elo+Features+Poisson → DB
<code>services/tournament_simulator.py</code>	Monte Carlo (vektorierte Gruppen + KO-Loop)
<code>services/tournament_projection.py</code>	Deterministische Einzelprojektion + <code>rank_and_pair</code>
<code>services/knockout_resolver.py</code>	KO-Teilnehmer aus realen Ergebnissen in DB schreiben
<code>services/bayesian_updater.py</code>	Elo-Update beider Teams nach Ergebnis (K=20, inkrementell)
<code>services/form_engine.py</code>	Form v2 (Punkte+Tore+Recency) — Single Source of Truth
<code>services/betting_engine.py</code> / <code>decision_engine.py</code>	EV, Edge, NO_BET, Best/Safe/Value
<code>services/odds_*</code>	Odds-Normalizer, Provider (Cache/Fallback), Aggregator, Calibration
<code>services/tipping_engine.py</code>	Punktoptimaler Tipp + <code>_points</code> (Kicktipp-Scoring)
<code>services/sync_service.py</code>	Idempotenter Sync von world-cup-2026-live-api
<code>routers/admin.py</code>	u. a. /auto-update (Sync + Elo + Recompute)

Datei	Rolle
<code>routers/matches.py</code>	Liste (verschlankt) + Detail + <code>_after_result_tasks</code>
<code>models/</code> <code>schemas/</code> <code>routers/</code>	SQLAlchemy-Models, Pydantic-Schemas, 7 Router
<code>scripts/</code> <code>alembic/</code>	seed/import/eval, Migrationen

Frontend (`frontend/src/`)

- Seiten: `/` (Dashboard), `/groups`, `/matches`, `/match/[id]`, `/bracket`, `/team/[id]`, `/tipps`
- `/tipps`: kommende Spiele + „Bereits gespielt“ (Tipp vs. Ergebnis + Punkte-Bilanz)
- `lib/tips.ts`: `computeTips` + `evaluateTip` (Kicktipp-Punkte); Match-Detail mit Tipp-vs-Ergebnis-Karte

CI (`.github/workflows/`)

- `keepalive.yml` — Health-Ping alle 10 min (Cold-Start)
- `sync-results.yml` — `/admin/auto-update` alle 30 min (Secret `ADMIN_TOKEN`)

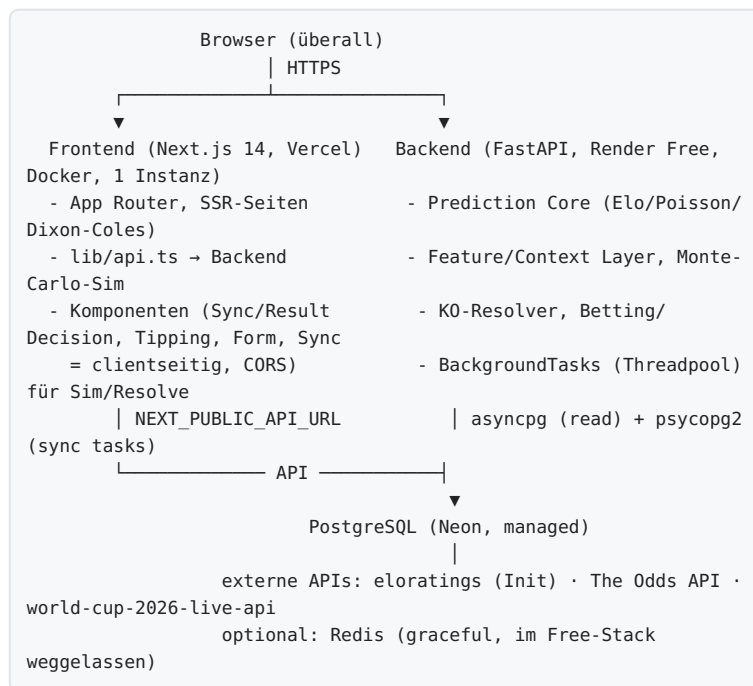
Datenbankstand (Neon, Stand vor WM-Start)

60 teams · 60 team_features · 48 group_memberships · 104 matches · 0 results · 72 predictions · 8 simulations.

Offene Punkte

Thema	Status
<code>ADMIN_TOKEN</code> als GitHub-Secret setzen (für Auto-Sync)	offen (User-Aktion)
Alten Render-Service <code>wm2026-backend</code> (US) löschen	optional (Aufräumen)
Venue-Zuordnung pro Spiel aus API	optional

ARCHITECTURE — Systemübersicht



Schichten (Backend)

1. **Data Layer** — SQLAlchemy-Models (`models/`), async Engine (`database.py`), Config (`config.py`).
2. **Domain/Services** (`services/`) — reine, gut testbare Logik (Elo, Poisson, Adjuster, Form, Simulator, Resolver, Betting, Odds, Tipping). Stateless wo möglich.
3. **Orchestrierung** — `prediction_engine` (async), Hintergrund-Tasks in `routers/matches`, `routers/admin.simulation_task`.
4. **API** (`routers/`) — 7 Router unter `/api/v1/`; Pydantic-Schemas (`schemas/`) als Contract.

Wichtige Design-Entscheidungen

- **Single-Instance-Backend:** In-Memory-Odds-Cache + BackgroundTasks ⇒ genau 1 Instanz/Worker.
- **Sim als sync BackgroundTask** → läuft im Starlette-ThreadPool, blockiert den Event-Loop nicht; HTTP-Request kehrt sofort (202) zurück.
- **Zwei DB-Treiber:** `asyncpg` (`DATABASE_URL`) für die App, `psycopg2` (`DATABASE_URL_SYNC`) für synchrone Hintergrund-Tasks/Scripts/alembic.
- **Schema:** `lifespan` ruft `create_all` (idempotent) beim Boot; alembic (001 Baseline aus Models, 002 Indizes) für reproduzierbare Migrationen.
- **Graceful Degradation:** Redis optional, Odds optional — System läuft immer.
- **Determinismus:** Prognose & Projektion deterministisch; Simulation reproduzierbar (Seed).

Datenfluss „Ergebnis eintragen“

Manuell: `POST /matches/{id}/result (ADMIN_TOKEN)`
→ MatchResult speichern + `status=FINISHED` → `apply_result` (Elo,

```
K=20)
  → BackgroundTask: form_engine → knockout_resolver → predict_all
  → cache.invalidate → _simulation_task

Automatisch (CI alle 30 min): POST /admin/auto-update
(ADMIN_TOKEN)
  → sync_all (API-Ergebnisse) → apply_result für Spiele OHNE
elo_ratings-Eintrag (idempotent)
  → derselbe BackgroundTask-Recompute
```

Automatisierung (GitHub Actions)

- `keepalive.yml` (10 min): GET `/health` → hält das Free-Backend wach (kein Cold-Start).
- `sync-results.yml` (30 min): POST `/admin/auto-update` (Secret `ADMIN_TOKEN`) → Auto-Sync.
- Extern getriggert, weil ein Render-Free-Service schläft und einen internen Scheduler mitnähme.

Frontend

Next.js 14 App Router. Datenseiten sind **Server Components** (SSR, holen die API serverseitig → kein CORS). Interaktive Teile (SyncButton, ResultForm) sind **Client Components** (Browser-Fetch → CORS).
`output: "standalone"`; auf Vercel nativer Next-Build.

Performance: ISR (`revalidate = 60`) cached die Seiten auf Vercel (stale-while-revalidate); `generateStaticParams` rendert alle `/match/[id]` + `/team/[id]` beim Build vor → sofortiger Erstauf. Backend in **Frankfurt** (zu Neon co-located) + verschlankte Queries + In-Process-Cache.

ML_MODEL — Modellarchitektur & Mathematik

Überblick: dreischichtiges Ensemble

```
EINZEL-SPIEL-PROGNOSE
  Elo (global, stabil)
    + Feature Adjustment Layer (deterministisch, 5 Faktoren)
      → angepasste Elo-Differenz → Poisson + Dixon-Coles → P(W/U/N)
    + xG + Scores

TURNIERSIMULATION (Monte Carlo)
  Elo (global)
    + Context Injection Layer (stochastisch, pro Run)
      → per-Run Elo-Variation → Poisson → Champion-/Runden-
        Wahrscheinlichkeiten
```

Kernprinzip: **Elo trägt die Vorhersage**, die Zusatzschichten verfeinern (deterministisch für Einzelspiele, stochastisch für Simulations-Varianz). Die Modell-Evaluation ([MODEL_EVALUATION.md](#)) belegt: Elo + Poisson + Dixon-Coles $\approx$ 99 % der Performance; die Features bewegen P(home) im Mittel nur ~ 2 pp.

Layer 1 — Elo Rating ([elo_model.py](#))

```
Erwartung:  $E(A) = 1 / (1 + 10^{((R_B - R_A)/400)})$ 
Update:  $R_{neu} = R_{alt} + K \cdot \text{Tordiff-Gewicht} \cdot (\text{Ergebnis} - \text{Erwartung})$ 
Tordiff-Gewicht:  $\Delta \leq 1 \rightarrow 1.00$ ,  $\Delta = 2 \rightarrow 1.50$ ,  $\Delta = 3 \rightarrow 1.75$ ,  $\Delta \geq 4 \rightarrow \min(1.75 + (\Delta - 3)/8, 2.50)$ 
K = 32 (reguläre Länderspiele), K = 20 (während WM, konservativer)
```

- **Initialwerte:** reproduzierbar aus **elratings.net** (gleiche Formulierung, SCALE=400); Import via `scripts/import_initial_elo.py` → `data/initial_elo.json` (`data_source='elratings_init'`). Beispiele: Spanien 2155, Argentinien 2113, Frankreich 2062, England 2020, Deutschland 1925.
- **Live:** Bayesian Update beider Teams nach jedem Ergebnis (`bayesian_updater.py`, K=20), Audit-Trail in `elo_ratings`. Elo ist die **alleinige dynamische Stärkemetri**k.

Layer 2a — Feature Adjustment

([feature_adjuster.py](#), Einzelspiel)

Jeder Faktor trägt einen Elo-Delta bei; der Gesamt-Delta (gedeckelt ± 150) wird hälftig auf beide Teams verteilt: `adj_home = elo_home + total*0.5`, `adj_away = elo_away - total*0.5`.

#	Faktor	Cap (Elo)	Quelle
1	Aktuelle Form	± 60	form_engine (echte Ergebnisse)
2	Marktwert	± 50	statischer PRIOR (Transfermarkt), eingefroren
3	Höhenunterschied	± 80	

#	Faktor	Cap (Elo)	Quelle
			Geodaten (Venue + Heimat)
4	Reisedistanz	±30	Geopy great_circle
5	Erholung (Rest Days)	±20	Spielplan (kickoff_utc), nur gespielte Spiele

Erholung: $\Delta \text{Elo} = 20 \cdot \tanh(\Delta d / 2.5)$ (Steigung ≈ 8 Elo/Tag, sanfte Sättigung). Aus `prediction_engine._rest_days` — zählt nur Spiele mit echtem Ergebnis → keine Phantom-Fatigue vor Turnierstart. **Entfernt** (Datenqualität): Alter, Caps (Seed-Schätzwerte, Einfluss ≈ 0), FIFA-Ranking (nur noch Elo-Init). Zeitzone nie implementiert.

Layer 2b — Context Injection (`context_modifier.py`, Simulation)

Stochastische Match-zu-Match-Varianz pro Run (deterministisch + Rauschen), gedeckelt ± 40 Elo:

Signal	deterministisch	σ stochastisch
Form-Momentum	±20 Elo (Form-Diff)	8
Kader-Tiefe (Marktwert)	—	5 (log-skaliert)
KO-Druck	—	4–6 (steigt mit Runde)
Umwelt-/Venue-Stress	—	bis 5 × Stress

Umwelt-Stress (`venue_environment_stress`): deterministischer Klima-/Venue-Proxy aus Höhe (ab ~ 1000 m) + Breitengrad (Juni/Juli-Hitze) $\in [0,1]$. **Nur Varianz, kein Bias** — der gerichtete Höheneffekt lebt in Layer 2a; kein Double-Counting (verschiedene Momente & Pfade).

Layer 3 — Poisson Goal Model (`poisson_model.py`)

```

λ_home = 1.30 · exp(elo_diff/800)      λ_away = 1.30 ·
exp(-elo_diff/800)    (clip 0.25–5.0)
P(i:j) = Poisson(i;λ_h) · Poisson(j;λ_a) · τ(i,j)    Grid 0..8,
dann normiert
Dixon-Coles τ (ρ=−0.13):  0:0→1−λhλap · 1:0→1+λap · 0:1→1+λhp ·
1:1→1−ρ · sonst 1
P(Heim)=Σ_{i>j} P(X)=Σ_{i=j} P(Ausw)=1−P(Heim)−P(X)

```

Monte-Carlo-Simulation (`tournament_simulator.py`)

- **Gruppenphase vektorisiert** (numpy): pro Gruppe alle Paarungen $\times n_{\text{runs}}$ Poisson-Samples, Tabellen-Ranking (Punkte→TD→Tore→Elo), Sieger/Zweite/beste 8 Dritte pro Run.
- **KO-Runden** als Python-Loop pro Run (R32→Finale), Paarung identisch zu `tournament_projection.rank_and_pair`; Elfmeter = $0.5 + (\Delta \text{elo})/10000$.
- Output: `champion_probs` + `stage_probs` je Team. Prod: **100.000 Runs** (`MONTE_CARLO_RUNS`), läuft als **Hintergrund-Task** (blockiert das Interface nicht). 30.000 Runs sind verifiziert gleichwertig (Spanien $\sim 18\%$) — als Spar-Option möglich, aber nicht nötig, da entkoppelt.

Kalibrierung

Backtest-Kriterium Brier/LogLoss/ECE (siehe MODEL_EVALUATION). ECE des Full-Modells ≈ 0.0016 (Poisson+Dixon-Coles liefern den größten Kalibrierungsgewinn).

MODEL_EXPLANATION — Das Modell in einfachen Worten

Für Tipprunden-Teilnehmer & Neugierige: *Wie kommt die Prognose zustande?* — ohne Formeln.

Die Grundidee

Jedes Team hat eine **Stärkezahl (Elo)**. Je höher, desto stärker. Spielen zwei Teams, vergleicht das Modell ihre Elo-Werte und leitet daraus ab, wie viele Tore jedes Team voraussichtlich schießt (**xG, erwartete Tore**). Aus diesen erwarteten Toren berechnet es die Wahrscheinlichkeit für **jedes mögliche Ergebnis** (0:0, 1:0, 2:1, ...) und fasst sie zu **Sieg / Unentschieden / Niederlage** zusammen.

Die Stärkezahl (Elo)

- Startwerte kommen von **elratings.net** (eine etablierte, öffentliche Welt-Rangliste) — keine Bauchentscheidungen.
- Nach **jedem echten Spiel** passt sich Elo an: Sieger steigen, Verlierer fallen; ein hoher Sieg zählt mehr als ein knapper, ein Sieg gegen einen Favoriten mehr als gegen einen Außenseiter.

Was die Prognose noch beeinflusst (kleine Korrekturen)

Zusätzlich zur Elo-Stärke berücksichtigt das Modell ein paar **Kontextfaktoren** — bewusst klein: - **Form** — wie ein Team zuletzt wirklich gespielt hat (Punkte + Tore der letzten Spiele). - **Marktwert** — grober Indikator für Kaderqualität. - **Höhe** — Spiele in großer Höhe (Mexiko-Stadt!) sind für Flachland-Teams zehrender. - **Reise** — sehr weite Anreise kostet etwas. - **Erholung** — wer mehr Pause seit dem letzten Spiel hatte, ist leicht im Vorteil.

Wichtig: Diese Faktoren **verschieben** die Prognose nur leicht (im Schnitt ~2 Prozentpunkte) — die **Elo-Stärke dominiert**. Das ist Absicht und durch Tests belegt.

Tore & Ergebnisse

Aus den erwarteten Toren erzeugt ein bewährtes statistisches Tormodell (**Poisson** mit **Dixon-Coles-Korrektur** für knappe Ergebnisse wie 1:0/1:1) die Wahrscheinlichkeit jedes Spielstands. Daraus kommen die Balken „Heimsieg/Unentschieden/Auswärtssieg“ und die Heatmap.

Der Tipp

- **xG-Tipp**: die gerundeten erwarteten Tore (z. B. xG 1.7:0.9 → „2:1“).
- **Modell-Tipp**: das wahrscheinlichste Einzelergebnis.
- **Punktoptimaler Tipp** (Tipps-Seite): nicht das wahrscheinlichste Ergebnis, sondern der Tipp, der über alle möglichen Ausgänge die **meisten erwarteten Tipp-Punkte** bringt.

Das ganze Turnier

Für Titelchancen wird das Turnier **zehntausende Male durchgespielt** (Monte-Carlo) — mit etwas Zufall pro Spiel (Tagesform, Nervenstärke, Bedingungen). Der Anteil der Durchläufe, in denen ein Team Weltmeister wird, ist seine **Titelwahrscheinlichkeit**.

Wetten (Decision Engine)

Wenn echte Buchmacher-Quoten vorliegen, vergleicht das System die **Modell-Wahrscheinlichkeit** mit der **fairen Markt-Wahrscheinlichkeit** (Buchmacher-Marge herausgerechnet): - **Edge** = wie viel optimistischer das Modell ist als der Markt. - **EV (Expected Value)** = lohnt sich die Wette rechnerisch? - Kategorien: **Value Bet** (Modell schlägt den Markt) bis **No Bet** (kein Vorteil). > Reines Analyse-Tool, kein Aufruf zum Glücksspiel.

Was das Modell NICHT weiß

- **Konkretes Wetter** (Regen/Wind/Tagestemperatur) — kein Forecast (nur ein grober Höhen-/Klima-Stress als Unsicherheit in der Simulation).
- **Verletzungen/Sperren** — kein Live-Datenstrom; Elo fängt es erst nach dem Spiel auf.
- **Taktik, Aufstellung, Schiedsrichter, Tagesform im Detail, Rivalitäten/Head-to-Head**. Diese Grenzen sind bewusst und dokumentiert ([LIMITATIONS.md](#)).

PREDICTION_LOGIC — Schritt für Schritt

Die Pipeline für eine Einzelspiel-Prognose
(`prediction_engine.predict_match`).

```
Input: match_id
1. Match + Teams + Venue + bisherige Prognosen laden
2. Elo beider Teams laden (neuester team_features-Eintrag;
Fallback 1500)
3. Rest-Days berechnen (nur GESPIELTE Vorspiele, aus kickoff_utc)
4. Feature-Adjustment (5 Faktoren → Elo-Deltas → angepasste Elo-
Werte)
5. Poisson + Dixon-Coles →  $\lambda$ , Score-Verteilung, P(W/U/N), Top-
Scorelines
6. Erklärung bauen (Faktoren mit Elo-Delta, Richtung, Summary)
7. Upsert in match_predictions (pro model_version)
Output: PredictionResult (Wahrscheinlichkeiten, xG, Scores,
Erklärung)
```

Schritt 2 — Elo

`elo = latest team_features.elo_rating` (sonst 1500). Typische Werte (eloratings, 06.06.2026): Spanien 2155 · Argentinien 2113 · Frankreich 2062 · England 2020 · Brasilien 1988 · Deutschland 1925 · USA 1733 · Saudi-Arabien 1569.

Schritt 3 — Rest-Days

`_rest_days(team, kickoff)`: jüngstes Spiel **mit Ergebnis** (`JOIN match_results`) vor `kickoff`. Tage = $(\text{kickoff} - \text{prev})/86400$. None = kein gespieltes Vorspiel → Faktor neutral.

Schritt 4 — Feature Adjustment

(`feature_adjuster.adjust`)

```
factors = {
    form:      cap((fh-fa)*50, 60),
    market:   cap(log10(mvh/mva)*45, 50),
    altitude:  Höhenstrafe(Venue, Heimathöhen), cap 80,
    travel:    Reisestrafе(great_circle), cap 30,
    rest:      cap(20*tanh( $\Delta d/2.5$ ), 20),      # nur wenn beide
rest_days bekannt
}
total = clip( $\Sigma$  factors,  $\pm 150$ )
adj_home = elo_home + total*0.5 ; adj_away = elo_away - total*0.5
```

Faktoren < Schwellwert (Höhe/Reise/Rest) werden weggelassen, damit nur relevante erscheinen.

Schritt 5 — Poisson + Dixon-Coles

```
 $\lambda_{\text{home}} = 1.30 \cdot \exp(\text{adj}\Delta/800)$  ;  $\lambda_{\text{away}} = 1.30 \cdot \exp(-\text{adj}\Delta/800)$  (clip 0.25–5.0)
 $P(i:j) = \text{Poisson}(i;\lambda_h) \cdot \text{Poisson}(j;\lambda_a) \cdot \tau(i,j)$   $i,j \in 0..8 \rightarrow$  normiert
 $P(\text{Heim}) = \Sigma_{\{i>j\}} \cdot P(X) = \Sigma_{\{i=j\}} \cdot P(\text{Ausw}) = \text{Rest}$ 
 $xG = \lambda_{\text{home}}/\lambda_{\text{away}}$  ; top_scorelines = Top-5 nach
Wahrscheinlichkeit
```

Turnier-Logik

- **Gruppe:** 3 Pkt Sieg / 1 Remis / 0; Tie-Break
Punkte→Tordifferenz→Tore→Elo.
- **Qualifikation:** Top 2 je Gruppe + 8 beste Drittplatzierte → 32 Teams (R32).
- **R32-Paarung** (kanonisch, `tournament_projection.rank_and_pair`):
A1-B2, A2-B1, C1-D2, ...
 - 8 beste Dritte paarweise.
- **KO:** Score aus Poisson; bei Gleichstand Elfmeter (≈50/50 + leichter Elo-Tilt).
- **KO-Befüllung in der DB:** `knockout_resolver.resolve_bracket`
schreibt nach Abschluss der Gruppenphase die R32-Teams, danach inkrementell R16...Finale aus realen KO-Ergebnissen → KO-Spiele bekommen Teams → `predict_all` erzeugt Prognosen → erscheinen in /tipps.

Auto-Update nach Ergebnis

(`matches._after_result_tasks`)

Ergebnis → Elo (Bayesian) → Form (`form_engine`) → KO-Bracket
(`resolver`) → `predict_all` → Cache invalidieren → Simulation neu. Der
Recompute-Teil ist idempotent.

Automatischer Ergebnis-Sync (`admin.auto_update` , CI alle 30 min)

`sync_all` (echte API-Ergebnisse) → für beendete Spiele **ohne**
`elo_ratings`-Eintrag (chronologisch) `apply_result` → derselbe
Recompute. Die „ohne Elo-Eintrag“-Bedingung macht das Ganze
idempotent (wiederholter Lauf wendet Elo nie doppelt an; manuell
eingetragene Spiele haben den Eintrag schon).

MODEL_EVALUATION — Ist die Komplexität gerechtfertigt?

Rigoreuse, deterministische Bewertung (fester RNG-Seed). Code: `scripts/eval/ ( metrics.py , run_evaluation.py )`. Ziel: trägt das Modell, oder reicht etwas Einfacheres?

Methodik

- **Varianten:** V1 Elo-only · V2 Elo+Poisson · V3 +Form · V4 +Form+Markt · V5 Full (DB-Prognose).
- **Bounded Backtest:** wenige reale Ergebnisse vor WM-Start → zwei klar benannte Ground-Truth-Szenarien (Full bzw. Elo+Poisson als „Wahrheit“), 2000 Outcome-Samples/Spiel.
- **Metriken:** LogLoss (primär), Brier, Accuracy, ECE (Kalibrierung).
- **Feature-Importance:** label-frei via Prognose-Sensitivität (mittlerer $|\Delta P(\text{home})|$ beim Permutieren eines Features).

Kernbefunde (72 Gruppenspiele, Seed 42)

Variante	LogLoss	ECE
Elo only	~1.05	~0.07-0.09
Elo + Poisson	~0.99	~0.002
+ Form / + Markt / Full	~0.98-0.99	~0.002

1. **Elo + Poisson + Dixon-Coles $\approx$ 99 % der Performance.** Der größte Sprung kommt vom Poisson-/Dixon-Coles-Layer (Kalibrierung: ECE $\sim 0.07 \rightarrow \sim 0.002$).
2. **Feature-Stack bewegt P(home) im Schnitt nur ~2 pp** und ändert nie die W/U/N-Reihenfolge.
3. **Feature-Importance:** Elo dominiert — ca. **6x stärker als Marktwert, ~14x stärker als Form**. Alter/Caps lagen darunter → **entfernt** (reines Rauschen aus Seed-Schätzwerten).
4. **Tipping:** EV-Maximierung schlägt argmax-Tipp deutlich ($\sim +24-28$ % erwartete Punkte).
5. **Konsistenz:** Monte-Carlo-Champion == deterministische Projektion (keine strukturelle Verzerrung).

Strategische Konsequenz

Das System ist **nicht mehr in der Modell-Entwicklungs-, sondern in der Datenqualitäts- und Entscheidungs-Phase**. Daraus folgten (alle umgesetzt): - Feature-Reduktion 7 $\rightarrow$ (4-)5 (Alter/Caps/FIFA-Live entfernt; Rest-Days als reproduzierbarer Faktor ergänzt). - Form vollständig datengetrieben (form_engine v2: Punkte + Tore + Recency). - Initial-Elo reproduzierbar aus `eloratings.net` statt Handwerten. - Fokus weiterer Arbeit auf den **Decision Layer** (EV + Edge gegen echte Quoten).

Verifikation nach Wiederherstellung (2026-06-07)

Nach dem Code-Rebuild reproduziert die Monte-Carlo-Simulation die bekannte Champion-Verteilung (Spanien ~ 18 %, Argentinien ~ 11 %, Frankreich ~ 7.6 %), ECE des Full-Modells gehalten (~ 0.0016), 0 Prognose-Inversionen über 72 Spiele.

Bekannte Grenzen der Evaluation

Wenige reale Ergebnisse vor WM-Start → label-freie Maße + bounded Backtest statt großem historischem Out-of-Sample-Test. Während des Turniers liefern reale Ergebnisse echte Labels; `run_evaluation.py` ist dafür wiederholbar.

DATA_SOURCES — Datenquellen & Ingestion

Übersicht

Datum	Quelle	Verwendung	Status
Spielplan, Tabellen, Ergebnisse	world-cup-2026-live-api (RapidAPI)	Sync (/wc/ draw , /wc/ standings)	aktiv (RAPIDAPI_KEY)
Initial-Elo	elratings.net (Datei-Import)	einmaliger Bootstrap	importiert
Wettquoten	The Odds API	Betting Decision Engine	aktiv (ODDS_API_KEY)
Form	berechnet aus match_results	form_engine v2	datengetrieben
Marktwert/Alter/ Caps	manueller Seed (Transfermarkt o. ä.)	PRIOR	statisch/ eingefroren
Venue-Geodaten	Seed (Höhe, lat/ lon)	Höhe/Reise/ Umwelt-Stress	statisch

Spieldaten — world-cup-2026-live-api

(sync_service.py)

- Host world-cup-2026-live-api.p.rapidapi.com , Header X-RapidAPI-Key / -Host .
- /wc/draw liefert die **72 Gruppenspiele** (Round 1-3) mit echten Kickoff-Zeiten; **keine** KO-Spiele (die KO-Termine sind Platzhalter / kommen aus dem offiziellen FIFA-Plan).
- sync_all(session) ist **idempotent**: aktualisiert Status/Kickoff, trägt Ergebnisse nach, legt nichts doppelt an. Namens-Mapping API→team_id wird aus der DB (home_country) gebaut
 - Aliase (z. B. „Bosnia & Herzegovina”→bosnia, „United States”→usa).
- /admin/status macht **keinen** API-Call (nur letzter bekannter Rate-Limit-Stand).
- **Automatisiert**: Der GitHub-Actions-Workflow sync-results.yml ruft alle 30 min POST /admin/auto-update → sync_all + Elo (idempotent für neue Spiele) + Recompute. Kein manuelles Eintragen nötig; POST /matches/{id}/result bleibt als Fallback. Der 30-min-Takt schont das RapidAPI-Kontingent (Rate-Limits werden getrackt, Graceful Fallback).

Initial-Elo — elratings.net

- Quelle committed: backend/data/elratings_2026-06-06.txt . Import: python scripts/import_initial_elo.py --file data/elratings_2026-06-06.txt --elratings --write-db → data/initial_elo.json + DB (data_source='elratings_init').
- Gleiche Elo-Formulierung wie das Projekt (SCALE=400) → keine Umrechnung, **kein manuelles Tuning**.
- Danach läuft Elo live über Bayesian Updates weiter.

Wettquoten — The Odds API (`odds_provider.py`)

- Sportkey `soccer_fifa_world_cup` , Märkte `h2h,totals` , Region EU, Decimal.
- TTL-Cache 15 min + **Fallback auf last-known odds** bei Rate-Limit/Fehler.
- Team-Matching: englische Namen (`home_country`) + Akzent-/Alias-Normalisierung.
- Ohne Key → graceful Fallback auf synthetische fair-odds (klar geflaggt, kein echter Edge).

Form — datengetrieben (`form_engine.py` , v2)

Ausschließlich aus realen Ergebnissen: Punkte (3/1/0) + Tordifferenz + Recency-Decay über die letzten N=6 Spiele → `form_score` ∈ [-1,1] .
Ohne Ergebnisse exakt 0.0. **Single Source of Truth** (nur diese Funktion + `scripts/refresh_form.py` schreiben `form_score`).

Statische PRIOR

- **Marktwert** — bewusst **nicht** automatisiert (Scraping fragil; Modellnutzen $6 \times < \text{Elo}$): statisch, eingefroren, klar gelabelt.
- **Alter/Caps** — als Modell-Faktoren entfernt; Spalten bleiben nur zur Anzeige.
- **Venue-Geodaten** — Höhe + Koordinaten; treiben Höhe/Reise (Adjuster) + Umwelt-Stress (Sim).

Fallback-Strategie

System funktioniert ohne externe APIs: DB-Stand bleibt erhalten, Ergebnisse manuell via `POST /matches/{id}/result` eintragbar, Elo/Form/Sim laufen lokal.

DATABASE_SCHEMA — PostgreSQL

Definiert in `backend/app/models/`. Schema entsteht via `create_all` (Boot) bzw. alembic (001 Baseline aus Models, 002 Indizes). Treiber: asyncpg (App) + psycopg2 (Tasks).

Tabellen

teams

`id` (PK, str) · `name` · `short_name` · `flag_emoji` · `confederation` · `home_country` (englisch, fürs Odds-/Sync-Matching) · `home_lat` · `home_lon` · `home_altitude_m` · `home_timezone` · `created/updated_at`.

team_features (UNIQUE: team_id, snapshot_date)

`id` (PK) · `team_id` (FK) · `snapshot_date` · `fifa_ranking` · `fifa_points` · **elo_rating** · `market_value_millions` · `avg_squad_age` · `avg_caps_per_player` · **form_score** · `form_goals_scored_avg` · `form_goals_conceded_avg` · **data_source** (`eloratings_init` / `form_engine_v2` / `seed_*` / `bayesian_update`) · `created_at`.

elo_ratings (Audit-Trail)

`id` (PK) · `team_id` (FK) · `rating` · `match_id` · `reason` · `created_at`. Index: (team_id, created_at).

groups · group_memberships

groups: `id` (PK, „A“...„L“) · `name`. group_memberships: (group_id, team_id) M:N.

venues

`id` (PK) · `name` · `city` · `country` · `altitude_m` · `lat` · `lon` · `timezone` · `capacity`.

matches

`id` (PK, z. B. `WC2026_<hash>` oder `WC2026_ROUND_OF_32_01`) · `tournament` · **stage** (`GROUP_STAGE` / `ROUND_OF_32` / ... / `FINAL` / `THIRD_PLACE`) · `group_id` (FK) · `match_number` · `home_team_id` (FK, **nullable** — KO-Platzhalter) · `away_team_id` (FK, nullable) · `venue_id` (FK) · `kickoff_utc` · **status** (`SCHEDULED` / `LIVE` / `FINISHED` / `POSTPONED`) · `created/updated_at`. Indizes: stage, status, kickoff_utc, (stage,status).

match_results

`match_id` (PK, FK) · `home_goals` · `away_goals` · `home_goals_ht` · `away_goals_ht` · `went_to_extra_time` · `went_to_penalties` · `penalty_winner_id` (FK) · `home_xg` · `away_xg` · `recorded_at` · `source`.

match_predictions (UNIQUE: match_id, model_version)

`id` (PK) · `match_id` (FK) · `model_version` · `predicted_at` · `prob_home_win/draw/away_win` · `xg_home` · `xg_away` · `top_scorelines` (JSONB) · `score_distribution` (JSONB) · `explanation` (JSONB) · `home/away_elo_at_prediction` · `home/away_features_snapshot` (JSONB). Index: match_id.

tournament_simulations

`id` (PK) · `model_version` · `n_runs` · `simulated_at` · **`champion_probs`**
(JSONB: `team_id`→p) · **`stage_probs`** (JSONB:
`team_id`→{`group_stage`,`round_of_32`,...,`champion`}) · `group_probs` ·
`tournament_state_hash` · `triggered_by`. Index: `simulated_at`.

Beziehungen (Kurz)

`team` 1—n `team_features` · `team` 1—n `elo_ratings` · `team` n—m `groups` ·
`group` 1—n `matches` · `match` 1—1 `match_result` · `match` 1—n
`match_predictions` · `match` n—1 `venue`.

Aktueller Stand (Neon)

60 teams · 60 `team_features` · 48 memberships · 104 matches · 0
results · 72 predictions · 8 simulations.

API_SPEC — REST-Endpunkte

Basis: `/api/v1`. Swagger UI: `/docs`. Live: `https://wm2026-backend-phwx.onrender.com`. Antworten als JSON. Admin-Endpunkte erfordern Header `Authorization: <ADMIN_TOKEN>` (auch `Bearer <token>` wird akzeptiert).

Health

- `GET /api/v1/health` → `{ "status": "ok" }`

Teams

- `GET /teams/` → `{ "teams": TeamSummary[] }` (sortiert nach Titelchance/Elo)
- `GET /teams/{id}` → `TeamDetail` (features, elo_history, tournament_probs, group_id)

Matches

- `GET /matches/?stage=&group_id=&team_id=&status=` → `{ "matches": MatchSummary[] }`
- `GET /matches/{id}` → `MatchDetail` (inkl. venue + PredictionDetail)
- `GET /matches/{id}/bets?`
`home_odds=&draw_odds=&away_odds=&over25_odds=&under25_odds=&bts_yes_odds=` → `BettingReport` (best/safe/value/no_bets, EV, Edge). Ohne Query-Quoten: echte Odds-API o. fair-odds.
- `GET /matches/{id}/tip` → punktoptimaler Tipp (`TipRecommendation`)
- `POST /matches/{id}/result (admin)` — Body `ResultCreate` (home_goals, away_goals, ...) → speichert Ergebnis, Elo-Update, startet Hintergrund-Kette (Form→KO→Predict→Sim).

Groups

- `GET /groups/` → `{ "groups": Group[] }` (Tabellen aus realen Ergebnissen + Qualifikations-%)

Tournament

- `GET /tournament/simulate (= /)` → `SimulationResult` (simulation_id, n_runs, model_version, simulated_at, champion_probabilities, stage_probabilities)
- `GET /tournament/champion-probabilities` → `{ "champion_probabilities": {team_id: p} }`
- `GET /tournament/projection` → deterministische Einzelprojektion (Tabellen, KO-Baum, Champion); Redis-gecacht
- `GET /tournament/bets?stage=&min_ev=&limit=` → aggregierte Best/Value Bets

Dashboard

- `GET /dashboard/` → `{ top_favorites[], tournament_status{matches_played,matches_total,stage}, next_matches[], recent_results[], last_simulation }`

Predictions

- `POST /matches/{id}/predict` → Prognose für ein Spiel (neu berechnen)
- `POST /predict-all` (*admin optional*) → alle SCHEDULED/LIVE-Spiele mit Teams

Admin (`Authorization-Header`)

- `GET /admin/status` → Sync-/Rate-Limit-Status (kein API-Call)
- `POST /admin/sync` → idempotenter Sync (world-cup-2026-live-api)
- `POST /admin/auto-update` → Sync + Elo idempotent für neu beendete Spiele + Recompute (Form/Bracket/Prognosen/Simulation). Von der CI (`sync-results.yml`) alle 30 min aufgerufen.
Antwort:

```
{ synced, elo_newly_applied, recompute: "triggered"|"skip ..." }
```
- `POST /admin/simulate` → Monte-Carlo im Hintergrund
- `GET /admin/simulation-status` → letzter Lauf
- `POST /admin/predict-all` ... (siehe Predictions)
- `POST /admin/refresh-form` · `POST /admin/refresh-features`
- `POST /admin/resolve-bracket` → KO-Teilnehmer setzen + neu prognostizieren
- `GET /admin/feature-audit` → COMPUTED/PRIOR-Klassifikation je Team
- `GET /admin/odds-status` → Key-/Fallback-Status, Alter der letzten Quoten, TTL
- `GET /admin/market-calibration` → Modell vs. Markt (KL, mittl. Edge, Brier)

Wichtige Response-Typen (Auszug)

- **TeamSummary:** id, name, short_name, flag_emoji, confederation, elo_rating, fifa_ranking, market_value_millions, avg_squad_age, form_score, champion_probability.
- **PredictionSummary/Detail:** prob_home_win/draw/away_win, xg_home/away, top_scoreline, (+ top_scorelines, score_distribution, explanation).
- **MatchSummary:** id, stage, group_id, home_team, away_team, kickoff_utc, status, prediction, result.

BETTING_ENGINE — Decision Layer (EV + Edge)

Wandelt Modell-Prognosen in strukturierte Wett-/Tipp-Empfehlungen.
Stateless, deterministisch, additiv — kein Eingriff in Elo/Poisson.
Reines Analyse-Tool.

Module

Modul	Rolle
<code>decision_engine.py</code>	Kanonische Facade (<code>compute_expected_value</code> , <code>compute_edge</code> , <code>rank_bets_by_ev</code> , <code>generate_recommendations</code>) — delegiert an <code>betting_engine</code>
<code>betting_engine.py</code>	Implementierung: <code>generate_report</code> , EV/Edge/Risiko, Best/Safe/Value/ No Bet
<code>odds_normalizer.py</code>	Single Source of Truth Overround- Entfernung (implizite & faire WS)
<code>odds_provider.py</code> / <code>odds_aggregator.py</code>	Odds-API-Client (Cache, last-known) + faire Markt-WS
<code>market_calibration_service.py</code>	Monitoring Modell vs. Markt (KL- Divergenz, Edge, Brier)

Kernkonzepte

```
EV  = Modell-WS × Quote - 1      # erwartete Auszahlung  
(hängt an der Quote)  
Edge = Modell-WS - faire Markt-WS  # Informationsvorsprung  
(overround-bereinigt)  
  
faire Markt-WS: p_raw = 1/odds ; overround = Σ p_raw ; p_fair =  
p_raw/overround (Σ=1)
```

- **EV** misst die Auszahlung, **Edge** die Überzeugung — sie ergänzen sich.
- Beispiel (echte Quoten): Heimsieg Modell 49.4 % vs. faire Markt-WS 45.4 % → Edge +4.0 pp, EV +3.7 %.

Empfehlungs-Kategorien (je Selektion

`recommendation`)

- **VALUE** — $EV \geq +2\%$ **und** positiver Edge.
- **SAFE** — hohe Wahrscheinlichkeit ($\geq 60\%$), geringes Risiko.
- **NO_BET** — negativer EV **oder** negativer Edge (Modell schlägt den Markt nicht).
- Report: `best_bet` (höchster EV), `safe_bet` , `value_bets` (Top-3),
 `no_bets` , `scoreline_tip` .

Märkte

1X2 · Over/Under 2.5 · BTTS · Correct Score (Top-3 aus Poisson). O/U & BTTS aus der `score_distribution` abgeleitet (nicht neu modelliert).

Quoten: echt vs. synthetisch

- **Echt** (Odds-API verfügbar): EV/Edge gegen den realen Markt; `odds_estimated=false`, `ev_calculable=true`. Team-Matching über englische Namen (DB `home_country`).
- **Synthetisch** (kein Key/Treffer): $\text{fair_odds} = (1/\text{WS}) \cdot (1 - \text{Marge})$; $\text{EV} \approx -\text{Marge} \rightarrow$ alles landet in `NO_BET` (kein echter Edge ohne echten Markt). Klar geflaggt.

Caching & Robustheit

TTL 15 min; bei API-Ausfall **last-known odds** statt Fehler; ohne Redis graceful (Odds-Cache ist in-memory pro Prozess $\rightarrow$ Single-Instance-Backend hält ihn warm).

Endpunkte

- `GET /api/v1/matches/{id}/bets` (optional echte Quoten als Query-Params)
- `GET /api/v1/matches/{id}/tip` (punktoptimaler Tipp, Tipping Engine)
- `GET /api/v1/tournament/bets?stage=...&min_ev=...` (aggregierte Value Bets)
- `GET /api/v1/admin/odds-status` · `GET /api/v1/admin/market-calibration`

Tipping Engine (`tipping_engine.py`)

Punktoptimaler Exakt-Tipp via **Erwartungswert-Maximierung** über die Score-Verteilung (nicht argmax). Kicktipp-Schema (`tipping_engine._points`): exakt 4 / Tordifferenz 3 (inkl. beide Remis) / Tendenz 2 / sonst 0. Frontend: `BettingPanel.tsx`, `TipPanel.tsx`.

Tipp-vs-Ergebnis-Auswertung (gespielte Spiele)

Für beendete Spiele wird der **empfohlene Tipp** (xG-Tipp, konsistent zur Tipps-Seite) gegen das **tatsächliche Ergebnis** ausgewertet und die erzielten **Kicktipp-Punkte** angezeigt. - Frontend-Logik `lib/tips.ts  $\rightarrow$  evaluateTip(tipScore, result)  $\rightarrow$  {points, kind, label}` — identische Punktelogik wie das Backend (Exakt 4 / Tordifferenz 3 / Tendenz 2 / 0). - Anzeige: **Tipps-Seite** Sektion „Bereits gespielt“ (pro Spiel Tipp $\rightarrow$ Ergebnis + Punkte) inkl. **Punkte-Bilanz** (Gesamtpunkte, exakt-Treffer, Trefferquote); **Match-Detail** als Vergleichskarte. - Reine Auswertung — keine Änderung am Prognose-/Tipping-Modell.

Haftungsausschluss

Statistisches Analyse-Tool, kein Wettangebot. Synthetische Quoten $\neq$ echte Buchmacher-Quoten. Keine Gewähr. Kein Aufruf zum Glücksspiel.

LIMITATIONS — Bekannte Grenzen (bewusst & dokumentiert)

Prinzip: lieber eine **explizit dokumentierte, reproduzierbare** Grenze als ein versteckter Schätzwert. Keine ML-Modelle, keine erfundenen Daten.

Im Modell enthaltene, aber bewusst begrenzte Faktoren

Thema	Umsetzung	Grenze
Form	form_engine v2 (Punkte+Tore+Recency, echte Ergebnisse)	vor 1. Spiel = 0.0 (neutral)
Rest-Days/Fatigue	nur gespielte Spiele, sanfte tanh-Sättigung (± 20)	greift erst während des Turniers; Gruppenphase weitgehend symmetrisch
Umwelt-/Venue-Stress	nur Varianz in der Simulation (Höhe+Klima-Proxy)	grober Proxy (Breitengrad statt echter Klimadaten); kein Bias, kein Forecast
Marktwert	statischer PRIOR, eingefroren	Importance $6 \times < \text{Elo}$; nicht automatisiert (bewusst)

Verbleibende, NICHT modellierte Faktoren

Thema	Impact	Mitigation
Verletzungen/Sperren	Hoch	kein reproduzierbarer Live-Stream; Elo fängt es nach dem Spiel auf; ggf. manuelle Elo-Korrektur
Konkretes Wetter (Regen/Wind/Temp)	Niedrig	kein Forecast (Reproduzierbarkeit); nur Umwelt-Stress als Varianz
KO-Sieger bei Elfmeterschießen	Niedrig	aus Torergebnis nicht ableitbar → Fallback höheres Elo (wie Sim/Projektion)
KO-Bracket-Pfad	Niedrig	vereinfachte A1-B2-Paarung, nicht der offizielle Überkreuz-Pfad
Taktik / Aufstellung / Schiedsrichter / Head-to-Head	mittel-niedrig	nicht modelliert; im Frontend als „Was das Modell nicht weiß“ kommuniziert
Venue-Zuordnung pro Spiel	Niedrig	vereinfacht (nicht je Spiel aus API); betrifft nur Höhe/Reise/Umwelt, alle klein gedeckelt
Dixon-Coles $p = -0.13$	Niedrig	aus Literatur (WM 2010–2022), nicht neu kalibriert

Bewusste Nicht-Ziele

Keine neuen ML-Modelle/Feature-Explosion (Prediction Core ist nahe optimal, siehe MODEL_EVALUATION). Keine nicht-reproduzierbaren Live-Quellen (z. B. Wetter-Forecast-APIs). Keine Änderung an Elo/Poisson/Dixon-Coles/Simulationsarchitektur ohne Anlass.

Betriebliche Grenzen (Free-Stack)

- Backend (Render Free) **schläft nach ~15 min** → Cold Start.
Gemindert durch den Keep-Alive-Workflow (`keepalive.yml` , Health-Ping alle 10 min); Detailseiten sind zudem vorgerendert (kein Backend-Treffer beim Erstaufruf).
- Backend **eine Instanz / ein Worker** (In-Process-Cache + BackgroundTasks) — bewusst, nicht skalieren.
- Simulation läuft als Hintergrund-Task (100k Runs; entkoppelt vom Interface); ein laufender Lauf geht bei Neustart verloren, ist aber idempotent neu auslösbar.
- Auto-Sync alle 30 min (RapidAPI-Kontingent) → Ergebnisse erscheinen mit bis zu ~30 min Verzug (kein Live-Ticker; bewusst, um die API-Quota zu schonen).

DEPLOYMENT — Free-Stack (Vercel + Render + Neon)

Live: Frontend <https://wm2026-prediction-system.vercel.app> · Backend <https://wm2026-backend-phwx.onrender.com> · DB Neon · Code github.com/annigue/wm2026-prediction-system. **Kosten:** 0 €/Monat.

Architektur

Komponente	Dienst	Plan / Region
Frontend (Next.js)	Vercel	Free
Backend (FastAPI, Docker)	Render	Free, Region Frankfurt (1 Instanz)
Datenbank (Postgres)	Neon	Free (Direct-Endpoint, eu-central, SSL)
Redis	—	weggelassen (graceful, In-Process-Cache statt)

`render.yaml` ist **Backend-only**; Frontend → Vercel, DB → Neon (extern).

Wichtig — Region: Backend und Neon liegen beide in **Frankfurt/eu-central**. Das eliminiert die Cross-Region-Latenz pro DB-Query (war die Hauptursache früherer Langsamkeit: Detailseite 4.5 s → 0.12 s). Render-Region ist pro Service fix → ein Wechsel erfordert Service-Neuanlage (daher die aktuelle URL mit `-phwx`-Suffix).

1. Datenbank (Neon)

1. neon.tech → Projekt (Region nahe Backend, **Frankfurt/eu-central**). **Direct connection** (Host **ohne** `-pooler`).

2. Migration:

```
pg_dump --no-owner --no-privileges --clean --if-exists <lokale-url> > wm2026.sql
psql "<neon-direct-url>" -f wm2026.sql
```

3. Treiber-URLs: `DATABASE_URL = postgresql+asyncpg://...?ssl=require` · `DATABASE_URL_SYNC = postgresql+psycopg2://...?sslmode=require` (asyncpg nutzt `ssl`, nicht `sslmode`).

2. Backend (Render, Blueprint)

1. Render → New → **Blueprint** → Repo, Branch `main`, `render.yaml` → Apply.
2. Env-Vars (sync:false): `DATABASE_URL`, `DATABASE_URL_SYNC`, `ODDS_API_KEY`, `RAPIDAPI_KEY`, `CORS_ORIGINS` (`*` oder Vercel-URL), `REDIS_URL` (leer). `ADMIN_TOKEN` generiert Render; `MONTE_CARLO_RUNS=100000` und `region: frankfurt` stehen in `render.yaml`.
3. Health `/api/v1/health` grün. Docker-Build (numpy/scipy) ~2–4 min.

3. Frontend (Vercel)

1. vercel.com → Add New Project → Repo. **Root Directory** = `frontend`.

2. Env-Var `NEXT_PUBLIC_API_URL` = Backend-URL (Build-Zeit → bei Änderung neu deployen!).
3. Deploy.

Frontend-Performance (im Code)

- **ISR** (`export const revalidate = 60 + fetch(..., {next:{revalidate:60}})`): Seiten aus Vercel-Cache.
- **generateStaticParams** für `/match/[id]` + `/team/[id]`: alle Detailseiten beim Build vorgerendert (Build ruft das Backend → Backend muss erreichbar sein; bei Ausfall Fallback auf On-Demand).

4. Automatisierung (GitHub Actions, `.github/workflows/`)

Workflow	Takt	Zweck
<code>keepalive.yml</code>	alle 10 min	<code>GET /health</code> — hält das Free-Backend wach (kein Cold-Start)
<code>sync-results.yml</code>	alle 30 min	<code>POST /admin/auto-update</code> — Ergebnisse syncen + Recompute

Secret: `sync-results.yml` braucht das Repo-Secret `ADMIN_TOKEN` (= Render- `ADMIN_TOKEN`). GitHub → Settings → Secrets and variables → Actions → New repository secret. Bei URL-Wechsel des Backends: `BASE` in beiden Workflows anpassen.

`/admin/auto-update` (idempotent)

1. `sync_all` → echte Ergebnisse von der WM-API in die DB.
2. Elo **nur für neu beendete Spiele** (die noch keinen `elo_ratings`-Eintrag haben), chronologisch.
3. Bei neuen Ergebnissen: Hintergrund-Recompute (Form → KO-Bracket → Prognosen → Cache → Simulation).

5. Migrations-/Safe-Deploy

- Schema: `create_all` (idempotent) beim Boot; Indizes via `alembic stamp 001 && alembic upgrade head`.
- Backend = 1 Instanz / 1 Worker (In-Process-Cache + BackgroundTasks) — nie skalieren.
- Rollback: Render Deploy-Rollback + Neon-Branch/Backup. Modellcode unverändert → kein Modell-Rollback.

6. Validierungs-Checkliste

- ☐ `/api/v1/health` 200, `/docs` lädt
- ☐ `/teams/` → 60, `/tournament/simulate` → `champion_probabilities`
- ☐ `/admin/odds-status` → `api_key_configured:true`
- ☐ Vercel: Startseite + Detailseiten schnell (`x-vercel-cache: HIT/PRERENDER`)
- ☐ GitHub Actions: `keepalive` grün; `sync-results` grün (nach `ADMIN_TOKEN`)
- ☐

Backend = 1 Instanz, Region Frankfurt

7. Betrieb

- Cold-Start durch keepalive weitgehend vermieden. 100k-Simulation läuft im Hintergrund.
- Render-Free-Kontingent: $24/7 \approx 720$ von 750 h/Monat — passt für den einen Service.
- Optional: `CORS_ORIGINS` exakt setzen, Neon-Passwort rotieren, alten US-Service löschen.

DECISIONS — Architecture

Decision Records (ADR)

Kompakte Begründungen der wichtigsten Entscheidungen.

ADR-001 — Elo als zentrale Stärkemetrik

Bewährt, interpretierbar, online-lernfähig. Live via Bayesian Update; alleinige dynamische Stärkequelle. *Alternative*: reine FIFA-Punkte (weniger prädiktiv) — verworfen.

ADR-002 — Poisson + Dixon-Coles statt direktem WDL-Klassifikator

Liefert volle Score-Verteilung (für xG, Heatmap, O/U, BTTS, Correct Score, Tipping) und den größten Kalibrierungsgewinn (ECE $0.07 \rightarrow 0.002$). Belegt in MODEL_EVALUATION.

ADR-003 — Feature-Layer additiv (Elo-Deltas), gedeckelt

Kontextfaktoren verschieben Elo um begrenzte Deltas (Gesamt ± 150), statt Wahrscheinlichkeiten direkt zu manipulieren $\rightarrow$ stabil & erklärbar. Einzelspiel deterministisch; Simulation stochastisch.

ADR-004 — Feature-Reduktion (7 $\rightarrow$ 4-5), datengetrieben

Evaluation: Elo dominiert (6 $\times$ Markt, 14 $\times$ Form). Alter/Caps (ungeprüfte Seed-Schätzwerte, Einfluss ≈ 0) **entfernt**; FIFA nur Elo-Init; Rest-Days als reproduzierbarer Faktor ergänzt.

ADR-005 — Initial-Elo aus eloratings.net

Reproduzierbar, gleiche Formulierung (SCALE=400), kein manuelles Tuning. Quelle committed; danach live via Bayesian.

ADR-006 — Form v2: Punkte + Tore + Recency, Single Source of Truth

Datengetrieben aus echten Ergebnissen, keine Seed-Schätzung, deterministisch. Genau eine schreibende Stelle (form_engine) $\rightarrow$ keine konkurrierenden Werte.

ADR-007 — Marktwert nicht automatisieren

Scraping fragil/ToS-kritisch; Modellnutzen $6\times <$ Elo. Statischer, eingefrorener PRIOR.

ADR-008 — Umwelt-/Venue-Stress nur als Varianz (kein Wetter-Forecast)

Reproduzierbarer Proxy (Höhe+Klima) erhöht nur die Simulations-Streuung, kein Bias; gerichteter Höheneffekt bleibt im Feature-Layer → kein Double-Counting. Forecast-APIs verworfen (nicht reproduzierbar).

ADR-009 — Rest-Days nur aus GESPIELTEN Spielen

Fatigue entsteht durch gespielte Spiele; Zählen terminierter Spiele erzeugte Phantom-Fatigue vor Turnierstart → Bug behoben (JOIN match_results) + sanfte tanh-Sättigung.

ADR-010 — KO-Bracket-Resolver (Persistenz statt nur In-Memory)

Projektion berechnete KO-Teilnehmer nur im Speicher → KO-Spiele blieben team-/prognoselos. Resolver schreibt qualifizierte Teams in die DB (R32 aus Tabellen, Folgerunden aus realen KO-Ergebnissen); Ranking-/Paarungslogik geteilt (rank_and_pair) — keine Duplikation.

ADR-011 — Decision Layer: EV und Edge

EV misst Auszahlung (quotenabhängig), Edge die overround-bereinigte Überzeugung. NO_BET bei negativem EV/Edge. Markt = informativer Prior, **nie** Override der Modell-WS.

ADR-012 — Single-Instance-Backend + sync BackgroundTasks

In-Memory-Odds-Cache + BG-Tasks ⇒ 1 Instanz/Worker. Sim als sync Task → Threadpool, blockiert den Event-Loop nicht. Bewusst kein Autoscaling.

ADR-013 — Free-Hosting-Stack (Vercel + Render-Free + Neon)

Kosten 0 € für ein privates Tool. *Alternativen:* alles-Render-paid / Oracle-VPS — situativ.

ADR-014 — Recovery-Strategie & .gitignore

Nach Quellcode-Verlust: Wiederherstellung aus .pyc-Decompile, **.next-Sourcemaps** (Frontend verbatim) und intakter DB; danach `.gitignore` (venv/.next/**pycache**/.env) + eigenes Repo, damit der Auslöser (Committen von Artefakten statt Source) nicht erneut passiert.

ADR-015 — Performance: Frankfurt-Co-Location + ISR + Prerender

Ursache früherer Langsamkeit war **nicht** das Modell, sondern Cross-Region-Latenz (Render-US ↔ Neon-EU) pro DB-Query × vieler selectinload-Round-Trips. Fixes: Backend nach **Frankfurt** (zu Neon co-located), Listen-Queries verschlankt (keine volle Prognose-JSONB/el_history), **In-Process-Cache** für die Projektion; im Frontend **ISR**

(`revalidate=60`) statt `no-store` + `generateStaticParams` (Detailseiten beim Build vorgerendert). Ergebnis: Detail 4.5 s → 0.12 s.
`MONTE_CARLO_RUNS=100000` bleibt (Sim ist Hintergrund-Task, entkoppelt vom Interface).

ADR-016 — Automatischer Sync via CI statt In-Process-Scheduler

Ein Render-Free-Service schläft → ein interner Scheduler stoppt mit ihm. Stattdessen triggert **GitHub Actions** extern: `keepalive.yml` (Health, 10 min) + `sync-results.yml` (`/admin/auto-update` , 30 min). Der externe Cron **weckt** das Backend zuverlässig. 30-min-Takt schont das RapidAPI-Kontingent.

ADR-017 — Auto-Update idempotent (Elo nur für Spiele ohne Elo-Eintrag)

Da der Sync wiederholt läuft, darf Elo nicht doppelt angewandt werden. `apply_result` ist inkrementell/nicht-idempotent → `/admin/auto-update` wählt nur beendete Spiele **ohne** `elo_ratings` -Eintrag (chronologisch). Manuelle Eingabe legt den Eintrag bereits an → wird übersprungen.

ADR-018 — Tipp-vs-Ergebnis transparent (Kicktipp-Scoring im Frontend)

„Empfohlener Tipp“ = xG-Tipp (konsistent zur Tipps-Seite). Punkte exakt zur Backend-Logik (`tipping_engine._points`): Exakt 4 / Tordifferenz 3 (inkl. Remis) / Tendenz 2 / 0. Anzeige auf Tipps-Seite (inkl. Punkte-Bilanz) + Match-Detail. Reine Auswertung, keine Modelländerung.

ROADMAP — Verlauf & Stand

Status: Feature-complete, evaluiert, gehärtet, **live deployed** (Free-Stack). WM-Start 2026-06-11.

Abgeschlossene Phasen

Phase	Inhalt
Architektur & Setup	FastAPI + SQLAlchemy async, Next.js 14, Docker, Modelle/Migrationen
Datenpipeline	world-cup-2026-live-api (RapidAPI) Sync; Seed
Elo + Poisson Modell	EloModel, PoissonModel + Dixon-Coles, FeatureAdjuster, PredictionEngine
Turniersimulation	Monte Carlo (vektorierte Gruppen + KO-Loop), Champion-/Runden-WS
UI/Dashboard	Dashboard, Gruppen, Spiele, Match-Detail (Heatmap/Erklärung), Team, Tipps, Bracket
Bayesian Updates	Elo-Update nach Ergebnis (K=20) + Hintergrund-Recompute
Context Injection	stochastische Simulations-Varianz (Form, Markt, KO-Druck)
Betting Decision Engine	EV, Edge, Risiko, Best/Safe/Value Bets, Märkte
Final Hardening	Form-Engine v2, Tipping-Engine (EV), Odds-Integration, Cache, Indizes
Model Evaluation	Ablation/Kalibrierung → Elo+Poisson ≈ 99 %; Empfehlungen
Decision Layer Opt.	Edge + Facade (decision_engine), Overround-Bereinigung
Data Quality Hardening	Form-Konsolidierung, 7→4 Faktoren, eloratings-Initial-Elo, Marktwert eingefroren
Hardening Pass	Rest-Days-Faktor, Umwelt/Venue-Stress-Varianz, LIMITATIONS
Hardening Fix-Pass	Rest-Days nur gespielte Spiele + tanh; „Wetter“→Umwelt-Stress; Double-Counting verifiziert
Odds-Markt-Upgrade	odds_normalizer (Single Source), NO_BET-Kategorie, last-known-Fallback, Calibration-Monitor
KO-Bracket-Resolver	KO-Teilnehmer aus realen Ergebnissen in DB → Prognosen/Tipps erscheinen automatisch
Recovery (2026-06-07)	Quellcode-Totalverlust → vollständig wiederhergestellt (siehe RECOVERY.md)
Deployment (2026-06-07)	Free-Stack live: Vercel + Render-Free + Neon
Doku-Neuaufbau (2026-06-08)	Gesamte docs/ nach dem Verlust neu erstellt
Performance (2026-06-08)	Frankfurt-Co-Location + ISR + generateStaticParams + verschlankte Queries + In-Process-Cache → Detail 4.5 s→0.12 s; Keep-Alive-Workflow

Phase	Inhalt
Auto-Sync (2026-06-08)	<code>/admin/auto-update</code> (idempotent) + <code>sync-results.yml</code> (30 min) → Ergebnisse organisieren sich automatisch
Tipp-Auswertung (2026-06-08)	Tipp vs. Ergebnis + Kicktipp-Punkte + Bilanz (Tipps-Seite & Match-Detail)

Risiko-Log (Auszug, alle behandelt)

- football-data.org deckt WM 2026 nicht ab → world-cup-2026-live-api ✓
- FIFA-Ranking doppelte Stärke zu Elo → nur Elo-Init ✓
- Form nicht datengetrieben / zwei Form-Engines → form_engine v2 als Single Source ✓
- Initial-Elo manuell getunt → eloratings.net-Import ✓
- KO-Spiele ohne Teilnehmer → knockout_resolver ✓
- Phantom-Fatigue vor Turnierstart → Rest-Days nur gespielte Spiele ✓
- **Quellcode-Verlust** → Recovery aus .pyc/.next-Sourcemap/DB ✓;
`.gitignore` verhindert Wiederholung
- Hosting-Kosten → Free-Stack ✓

Offen / Optional

- Uptime-Pinger gegen Cold Start · CORS exakt · Neon-Passwort rotieren · Venue-Zuordnung pro Spiel aus API.

RECOVERY — Quellcode-Wiederherstellung (2026-06-07)

Was passiert war

Der gesamte Quellcode unter `2026-06 WM 2026/` war von der Platte verschwunden. Übrig blieben nur kompilierte Artefakte: `backend/**/__pycache__/*.pyc`, `frontend/.next/`, `venv/`, `backend/.env`. Ein „Initial commit“ hatte fälschlich nur diese Artefakte (Bytecode/venv) erfasst, **nicht** den Quellcode (`.py`, `.tsx`, `.md`, Configs). Ursache war ein Commit-/Clean- Ablauf, der Source-Dateien verwarf und Build-Artefakte einbezog.

Die PostgreSQL-DB war vollständig intakt — der wertvollste Teil (alle Daten) war nie betroffen.

Wiederherstellungsquellen

Bereich	Quelle	Fidelität
Backend <code>.py</code>	Decompile der <code>.pyc</code> (Python 3.12, <code>pycdc</code>) + Session-Wissen	hoch (Logik exakt; Deklaratives 1:1, Methodenkörper rekonstruiert)
Frontend <code>.tsx/.ts</code>	<code>.next eval-source-map</code> → eingebettete <code>sourcesContent</code>	verbatim (Originalquelle!)
Config/Infra/Docs	Session-Wissen + DB + <code>pip freeze</code>	rekonstruiert
Initial-Elo-Daten	vom Nutzer eingefügter <code>elratings-Export</code>	exakt
Daten (Teams/Spiele/...)	intakte DB	exakt

Vorgehen (Batches, jeweils verifiziert)

1. **Backup** aller Recovery-Assets (read-only): `/home/anne/wm2026_recovery_backup_20260607_130233`.
2. **Decompiler** (`pycdc`) gebaut; alle `.pyc` → Referenz.
3. Backend in Schichten: Data-Layer → Core-Services → Logik-Services → Simulator (gegen bekannte Champion-Verteilung verifiziert) → Router + `main` (voller App-Boot gegen DB, alle Endpoints 200).
4. Scripts + `alembic` + Infra (`requirements.txt` aus `venv`); `.gitignore`.
5. **Frontend** aus `.next`-Sourcemaps extrahiert (24 Dateien verbatim) + Configs/ `types.ts` neu; `npm run build` erfolgreich. Die wiederhergestellte `api.ts` diente als maßgeblicher API-Contract → Backend exakt darauf ausgerichtet.
6. Sauberes, **eigenes** Git-Repo angelegt, vollständigen Source committet (diesmal echte Quellen, keine Artefakte), zu GitHub gepusht.
7. Deploy auf Free-Stack (Vercel + Render-Free + Neon), DB migriert, end-to-end verifiziert.

Lessons learned / Prävention

- `.gitignore` schließt jetzt `venv/`, `node_modules/`, `.next/`, `__pycache__/`, `.env` aus — der Auslöser (Artefakte committen, Source ignorieren/löschen) kann nicht erneut greifen.

- **Eigenes Repo** für das Projekt (nicht im fremden „Albumgenerator“-Repo).
- Dev-Builds mit `eval-source-map` enthalten den Original Quellcode — ein realer Recovery-Anker für Frontends ohne Bytecode.
- Reproduzierbare externe Daten (elratings-Datei committed) erleichtern Wiederaufbau.

Verbleibende Artefakte

- Backup: `/home/anne/wm2026_recovery_backup_20260607_130233` (pyc, .next, .env, DB-Dump).
- Decompile-Referenz: `/home/anne/wm2026_recovery_work/decompiled`.
- DB-Migrationsdump: `/home/anne/wm2026_neon_migration.sql`.